

INTRODUCTION TO ROBOTICS

MIHIR RAO

Contents

1	Intro to Robotics	2
1.1	Motivation and scope	2
1.2	What counts as a robot?	2
1.3	The sensing–computation–action loop	3
1.4	Three connected technical areas	3
1.5	Uncertainty as a common obstacle	4
2	Discrete planning (graph search)	5
2.1	The geometric planning model	5
2.2	Configuration space	5
2.3	From a grid to a graph	6
2.4	Forward search and breadth-first search	7
2.5	Depth-first search	8
2.6	Path costs and A* priorities	8
2.7	A* updates and implementation	9

LECTURE 1: INTRO TO ROBOTICS

1.1 Motivation and scope

Robotics uses computation to act in the physical world. Examples include household robots, walking robots, self-driving cars, and delivery drones. The goal is to make machines perform useful tasks reliably in real settings.

These systems combine machine learning, control, optimization, and computer vision. Using them in practice also raises questions about privacy, jobs, safety, and whether the robot does what people intend.

1.2 What counts as a robot?

Modern robotics belongs to a long history of imagined artificial beings and mechanical automata, from the bronze giant Talos of Greek mythology through the automata of antiquity, the Banu Musa brothers, Ismail al-Jazari, Leonardo da Vinci, and Descartes, to the intricate automata of the eighteenth century and Charles Babbage's calculating engines (figure 1). Two landmarks are Karel Čapek's 1920 play *R.U.R. (Rossum's Universal Robots)*, which introduced the word "robot" (from a Czech word for work or labor), and the development of the Unimate industrial manipulator by George Devol and Joseph Engelberger in the 1950s.

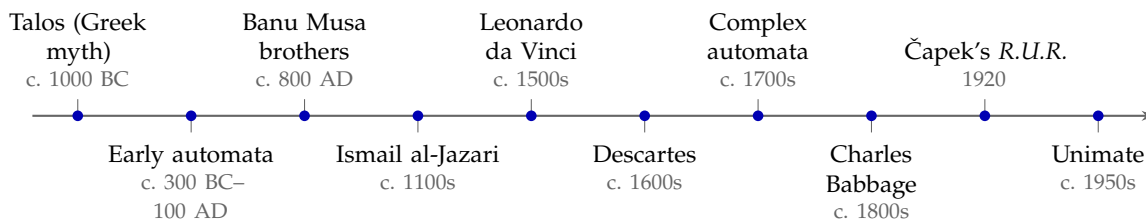


Figure 1: Milestones in the prehistory of robotics, in chronological order. Spacing is ordinal, not proportional to time.

Definition: A **ROBOT** is an embodied agent that can be programmed to perform physical tasks. Here, **EMBODIMENT** means having a physical body that interacts with the world: a robot's program produces physical actions, not just information.

Definitions differ in what they require: automatic actions, reprogrammable movement, or sensing and decision-making. The choice affects which machines count as robots. A dishwasher and an automatic door test how broad a definition should be; a surgical system controlled by a person tests whether autonomy is essential. A humanoid shape is not required, and programmability does not by itself imply independence from human supervision.

Note. There is no universally sharp boundary between robots and other machines. The working definition helps organize the subject; it is not a test that settles every borderline example.

1.3 The sensing–computation–action loop

A robotic system combines three functions. A **SENSOR** measures aspects of the robot or its surroundings. **COMPUTATION** converts measurements and task information into decisions. An **ACTUATOR** turns commands into physical effects, such as the thrust produced by a drone’s motor-driven rotors. The resulting motion changes what the sensors observe next, closing the cycle shown in [figure 2](#), often summarized as sense–think–act.

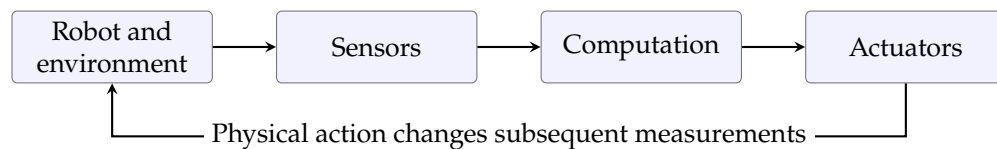


Figure 2: The sensing–computation–action loop. Sensors measure the robot and its environment, computation turns measurements into commands, and actuators carry out physical actions that change later measurements.

A small quadrotor is a concrete example: its sensors include an inertial measurement unit and an optical-flow camera, its onboard computation processes information, and its rotors provide actuation. A **FEEDBACK** loop uses new measurements to adjust subsequent actions.

1.4 Three connected technical areas

PLANNING selects a course of action toward a goal; **CONTROL** determines how to execute and adjust that action. **GEOMETRIC MOTION PLANNING** focuses on the spatial feasibility of motion, including obstacle avoidance. **DYNAMICS** describes how the robot’s motion evolves under applied inputs. The topics build in sequence: geometric planning, robot dynamics, feedback control, and planning subject to those dynamics.

STATE ESTIMATION uses measurements to estimate quantities such as the robot’s position and velocity. **LOCALIZATION** estimates where the robot is, while **MAPPING** estimates the surrounding environment. Standard estimation methods include Bayes filtering, Kalman filters, and particle filters. **SIMULTANEOUS LOCALIZATION AND MAPPING (SLAM)** addresses localization and map construction together.

ROBOT LEARNING uses data or experience to improve robotic behavior. Examples include imitation learning, generative models, reinforcement learning, and world models. Each uses data differently to guide actions.

1.5 Uncertainty as a common obstacle

UNCERTAINTY is incomplete or imperfect knowledge relevant to a decision. It arises in all three technical areas:

Area	Sources of uncertainty
Planning and control	Robot dynamics, changes in the world, and the robot's initial condition.
Localization and mapping	The robot's state, environmental geometry, and the meaning of objects or observations.
Robot learning	Data quality, human intent, and the rewards supplied by interaction with the world.

Example: For a drone navigating around obstacles, a proposed route is useful only if the drone can execute it and estimate where it is. An imperfect motion model affects execution, while an inaccurate position estimate affects both control and obstacle avoidance. This illustrates why the three areas must work together.

Source: Anirudha Majumdar, *Introduction to Robotics*, Princeton University, ROB 345 / 549, Lecture 1, "Introduction to Robotics," [lecture slides](#), PDF pages 1–57. [Official course materials](#); accessed September 19, 2026. The explanations, figures, and drone example are original.

LECTURE 2: DISCRETE PLANNING (GRAPH SEARCH)

2.1 The geometric planning model

Geometric planning finds a route through a known environment without collisions. This is the classical PIANO MOVER'S PROBLEM, in which both the position and orientation of an extended object can matter. General formulations can be PSPACE-complete; practical methods therefore rely on useful representations and search strategies.

For now, assume that the environment map is given, the robot's configuration is known exactly, and every geometric path is feasible to execute. The last assumption sets aside dynamics: a collision-free route need not yet respect acceleration limits or the forces needed to follow it. Planning also appears outside navigation, for example when searching possible shapes of a molecule.

2.2 Configuration space

A CONFIGURATION describes the robot's position and arrangement. The DEGREES OF FREEDOM (DoFs) of a system are the number of independent ways it can move, and its CONFIGURATION SPACE C is the set of possible configurations it can be in. In this geometric model, these coordinates describe arrangement, not velocity.

System	DoFs	Configuration space
Point in d dimensions	d	$\mathbb{R}^d$
N independent points in d dimensions	Nd	$\mathbb{R}^{Nd}$
Pendulum with unrestricted angle	1	S^1
Planar quadrotor	3	$\mathbb{R}^2 \times S^1$

Here S^1 is the circle: angles differing by 2π describe the same orientation. For the planar quadrotor, $q = (x, y, \theta)$ combines two position coordinates with one orientation. Independent coordinates need not all have the geometry of an unbounded real line.

The FREE CONFIGURATION SPACE $C_{\text{free}} \subseteq C$ contains configurations without collision ([figure 3](#)). Its complement, OBSTACLE CONFIGURATION SPACE, is

$$C_{\text{obs}} = C \setminus C_{\text{free}}.$$

The planning problem is to find a continuous path

$$\gamma : [0, 1] \longrightarrow C_{\text{free}}, \quad \gamma(0) = q_A, \quad \gamma(1) = q_B.$$

An obstacle in configuration space represents a forbidden arrangement of the whole robot. Checking only the position of a robot's center would ignore its extent and, when relevant, its orientation.

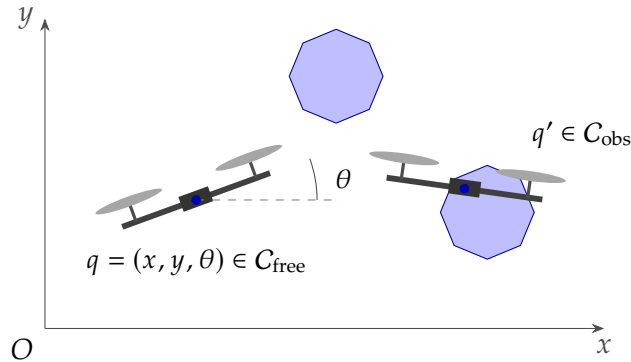


Figure 3: A planar quadrotor has configuration $q = (x, y, \theta)$: the position of its center and its tilt angle. With the two octagonal obstacles shown, the left configuration is collision-free, while the right one overlaps an obstacle and so lies in C_{obs} .

2.3 From a grid to a graph

DISCRETIZATION replaces the continuous problem with candidate configurations and transitions. A GRAPH $G = (V, E)$ records them as vertices V and edges E . Each grid cell supplies a vertex; adjacent cells are connected, and obstacle vertices and their incident edges are removed (figure 4).

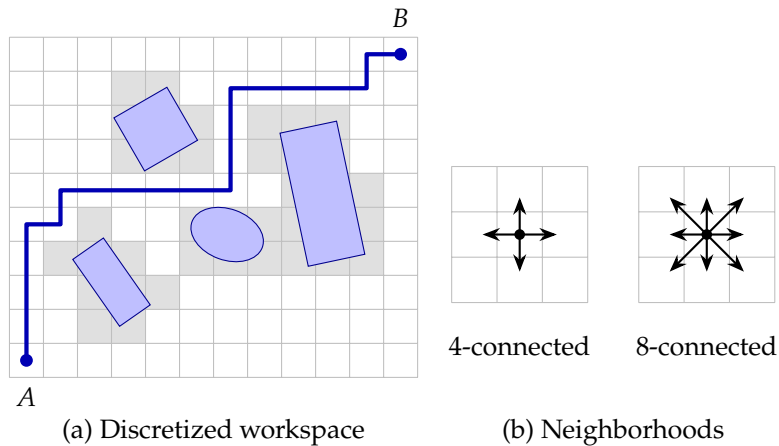


Figure 4: (a) A uniform grid over a workspace with four obstacles. Shaded cells touch an obstacle and are removed; the route from A to B uses only horizontal and vertical moves between free cells. (b) The moves allowed from a cell in a 4-connected and an 8-connected grid.

A 4-CONNECTED GRID permits horizontal and vertical moves. An 8-CONNECTED GRID also permits diagonal moves. The grid resolution and connectivity are modeling choices: a finer grid represents geometry more accurately but produces a larger search problem. A coarse grid can erase a narrow passage, so failure on the graph does not establish that no continuous path exists.

Note. The graph should represent collision-free motion, not just collision-free endpoints. Account for the robot's size when constructing obstacle cells, and check the transitions themselves; diagonal moves must not cut through obstacle corners. A safe endpoint alone does not make the move safe.

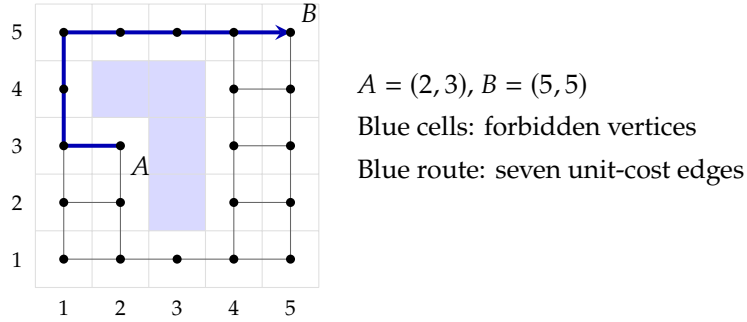


Figure 5: The running example: a 5×5 4-connected grid whose shaded cells are blocked. Vertices are labeled (column, row), and the highlighted route is a shortest path from A to B .

2.4 Forward search and breadth-first search

FORWARD SEARCH explores reachable neighbors starting at A . Its FRONTIER Q stores discovered vertices waiting to be processed. A vertex's PARENT records which vertex first led us to it. Following these links backward recovers the path.

The basic procedure is:

1. Insert A into Q and mark it discovered.
2. Remove a vertex x from Q . If x is a goal, recover its path and stop.
3. For each undiscovered neighbor y , set $\text{parent}(y) = x$, mark y discovered, and insert it into Q .
4. Repeat until a goal is removed or the frontier is empty. An empty frontier means that no goal is reachable in this graph.

Marking a vertex on insertion prevents repeated insertion through another edge or a cycle. To recover a path, follow parent pointers from the goal back to A and reverse the resulting sequence. A choice of frontier removal rule turns this procedure into different search algorithms.

BREADTH-FIRST SEARCH (BFS) uses a first-in, first-out queue: the oldest frontier vertex is processed next. Vertices are explored in layers of increasing number of edges from the start. For the graph in [figure 5](#) with a fixed neighbor order, the first few queue states are:

Vertex just processed	Queue afterward, next removal on the left
$A = (2, 3)$	$[(1, 3), (2, 2)]$
$(1, 3)$	$[(2, 2), (1, 4), (1, 2)]$
$(2, 2)$	$[(1, 4), (1, 2), (2, 1)]$

Proposition 2.1 [BFS SHORTEST PATHS]. *On a finite graph, BFS finds a reachable goal using the fewest edges. This is also a minimum-cost path when every edge has the same positive cost.*

Proof. The queue processes every vertex at distance k before any vertex at distance $k + 1$. A vertex first discovered from layer k therefore has a path with $k + 1$ edges and cannot have an undiscovered shorter path. Applying this to the first goal processed proves the claim. $\square$

With adjacency lists and discovery checks, each vertex is inserted once and each edge is examined a bounded number of times, giving $O(|V| + |E|)$ time and $O(|V|)$ auxiliary storage. The grid can still make $|V|$ and $|E|$ very large.

2.5 Depth-first search

DEPTH-FIRST SEARCH (DFS) instead uses a last-in, first-out stack: the most recently inserted vertex is processed next. It pursues a branch until it reaches the goal or runs out of new neighbors, then resumes another pending branch. Neighbor order matters because it determines which branch is tried first.

If the neighbors of A are inserted as $(2, 2)$ and then $(1, 3)$, then $(1, 3)$ is the next vertex processed. DFS continues up the left side and across the top row, reaching B before exploring the remaining lower branch. Such behavior can be useful when a solution is deep and there are few promising branches.

On a finite graph, the discovered set still makes the search terminate, with $O(|V| + |E|)$ time and up to $O(|V|)$ stored information. DFS does not generally return a shortest path; finding a short route in this particular example is not an optimality guarantee.

2.6 Path costs and A^* priorities

BFS minimizes the number of edges, not arbitrary travel costs, and may explore many vertices that do not lead toward the goal. Assign each edge (x, y) a nonnegative EDGE COST $\ell(x, y)$. The PATH COST for $p = (x_0, \dots, x_k)$ is

$$J(p) = \sum_{i=0}^{k-1} \ell(x_i, x_{i+1}).$$

The objective is now to minimize $J(p)$ among paths from A to B .

A* SEARCH orders the frontier using

$$F(x) = C(x) + H(x),$$

where the COST-TO-COME $C(x)$ is the cheapest cost to x found so far, and the HEURISTIC $H(x)$ estimates the remaining cost to the goal. Thus $C(x)$ is the cost of a known path, not necessarily the optimal cost to x yet. The frontier is a PRIORITY QUEUE, which removes a vertex with the smallest F value.

An ADMISSIBLE HEURISTIC never exceeds the true minimum remaining cost $H^*(x)$: $0 \leq H(x) \leq H^*(x)$, with $H(B) = 0$. Ignoring obstacles often produces a useful lower bound. Euclidean distance is valid when each edge's cost is at least its Euclidean displacement. Setting $H \equiv 0$ gives DIJKSTRA'S ALGORITHM, which prioritizes cost-to-come alone.

For the 4-connected grid of [figure 5](#) with unit edge costs, the MANHATTAN DISTANCE

$$H(x) = |x_1 - 5| + |x_2 - 5|$$

counts the horizontal and vertical moves needed without obstacles. Obstacles can require detours but cannot reduce that count. Initially $C(A) = 0$, $H(A) = 5$, and $F(A) = 5$. The first updates are:

Newly reached vertex	C	H	F
(1, 3), from A	1	6	7
(2, 2), from A	1	6	7
(1, 4), from (1, 3)	2	5	7
(1, 2), from (1, 3)	2	7	9
(1, 5), from (1, 4)	3	4	7

The first two candidates tie in both F and H ; either may be chosen. Later, breaking an F tie by smaller H favors (1, 4) over (2, 2) and directs search along the upper route. The illustrated solution costs 7, exceeding the initial lower bound 5 because of the obstacle.

Heuristics must match the motion and cost model. Manhattan distance can overestimate when diagonal moves are allowed cheaply. Search heuristics can also be learned from data, but a learned estimate does not automatically satisfy the lower-bound condition required for an optimality guarantee.

2.7 A* updates and implementation

It helps to distinguish undiscovered vertices, OPEN VERTICES (alive, stored in the frontier), and CLOSED VERTICES (dead, already expanded). Initialize $C(A) = 0$ and $C(x) = \infty$ otherwise. When expanding x , compute the candidate cost $C(x) + \ell(x, y)$ for each neighbor y . If it improves $C(y)$, update $C(y)$, its parent, and its priority. This operation is RELAXATION. Unlike BFS discovery marking, first

encountering a vertex does not settle its cheapest path.

Note. **When closing a vertex is permanent.** A CONSISTENT HEURISTIC satisfies $H(x) \leq \ell(x, y) + H(y)$ on every edge, with $H(B) = 0$. Consistency ensures that closing a vertex does not discard a later cheaper route. Admissibility alone does not justify this rule: with an inconsistent heuristic, improved paths may require reopening closed vertices. Manhattan distance on this unit-cost, 4-connected grid is consistent.

The implementation below never reopens a closed vertex. It assumes a finite graph, nonnegative costs, and a consistent nonnegative heuristic that is zero at the goal. The function `neighbors(x)` yields pairs (vertex, edge_cost). Improved priorities are pushed as new heap entries; obsolete entries are ignored when removed. A counter breaks any remaining ties, so Python does not need to compare vertex names.

Python implementation

```
1 from heapq import heappop, heappush
2 from itertools import count

3 def astar(start, goal, neighbors, heuristic):
4     serial = count()
5     h0 = heuristic(start)
6     queue = [(h0, h0, next(serial), 0, start)]
7     cost, parent, closed = {start: 0}, {}, set()
8     while queue:
9         _, _, _, g, x = heappop(queue)
10        if x in closed or g != cost[x]:
11            continue
12        if x == goal:
13            path = [x]
14            while x != start:
15                x = parent[x]
16                path.append(x)
17            return path[::-1], g
18        closed.add(x)
19        for y, edge_cost in neighbors(x):
20            if y in closed:
21                continue
22            candidate = g + edge_cost
23            if candidate < cost.get(y, float("inf")):
24                cost[y], parent[y] = candidate, x
25                h = heuristic(y)
26                heappush(queue, (candidate + h, h,
27                                next(serial), candidate, y))
28    return None
```

Stop when the goal is removed as the best valid frontier entry, not when it is first discovered. If the frontier becomes empty, no path exists in the graph. Even a minimum-cost graph path is optimal only for the chosen discretization, edge costs, and geometric assumptions.

Source: Anirudha Majumdar, *Introduction to Robotics*, Princeton University, ROB 345 / 549, Lecture 2, “Discrete Motion Planning,” [lecture slides](#), PDF pages 1–79, listed as “Discrete planning (graph search)” on the [official course page](#); accessed September 20, 2026. The figures, proof, and Python implementation are original explanatory reconstructions. The consistency qualification is supported by David L. Poole and Alan K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed., [Section 3.7.2](#).